

AI-Based Restoration of Degraded Images for Semiconductor Inspection

PHASE 2

A 6.81 Million parameter MiniRestormer, a channel-attention transformer U-Net that predicts a correction on top of a bicubic upsample that turns a 128×128 speckled input into a 256×256 restoration in a **single forward pass**.

TEAM

Git Push and Pray

Kalyan S | R Kiruthika | Sanjay G
Indian Institute of Technology

REPO

https://github.com/KAlcorpse/Git_Push_and_Pray_Phase_2

Problem understanding and the restoration task

What we are given

- 4,785 paired images: clean GT 256×256 in [0,1], degraded NoisyLR 128×128
- NoisyLR deliberately escapes [0,1] measured -0.28 to +2.24
- Speckle, Poisson shot, additive read noise, then downsampling, the order is not disclosed

What we must produce

- A standalone script: input dir to output dir, same filenames, $2H \times 2W$
 - Two scales: 128→256 and 256→512
- Generalise to unfamiliar image content, not just unfamiliar noise

How it is scored

- A fixed weighted combination of PSNR, SSIM and LPIPS wherein the weights are undisclosed
 - End-to-end H100 time: faster preferred when quality is comparable
 - Test set has OOD samples from different sources; evaluates accuracy AND robustness

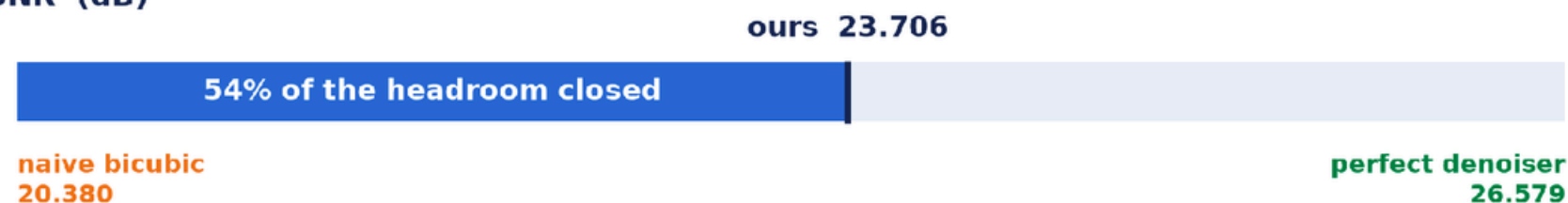
Our reading of the task: *This is a denoising problem wearing a super-resolution costume.*

A perfect denoiser followed by plain bicubic scores **26.579 dB**, so the 2× upsample costs almost nothing and essentially all the headroom is in removing speckle. That measurement set our priorities before we trained anything.

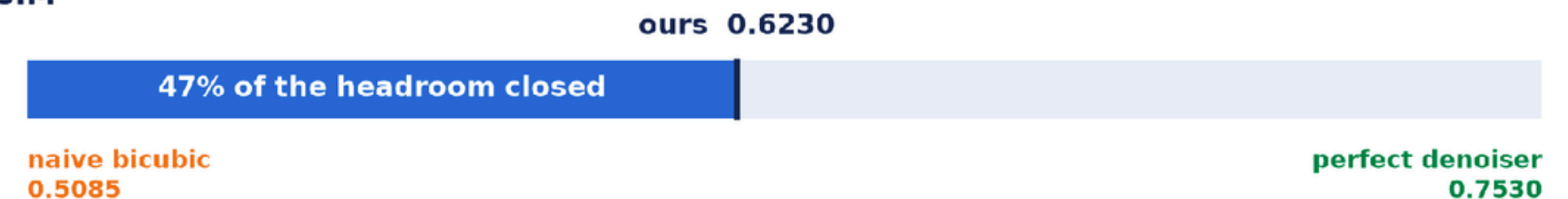
We closed **54% of the PSNR headroom** and **47% of the SSIM headroom** on validation set and **53% of the PSNR** and **48% of the SSIM** in the later released test set

479 held-out images. Both endpoints measured on the same images, not assumed. On the 297 released test images: 52% and 48%.

PSNR (dB)



SSIM



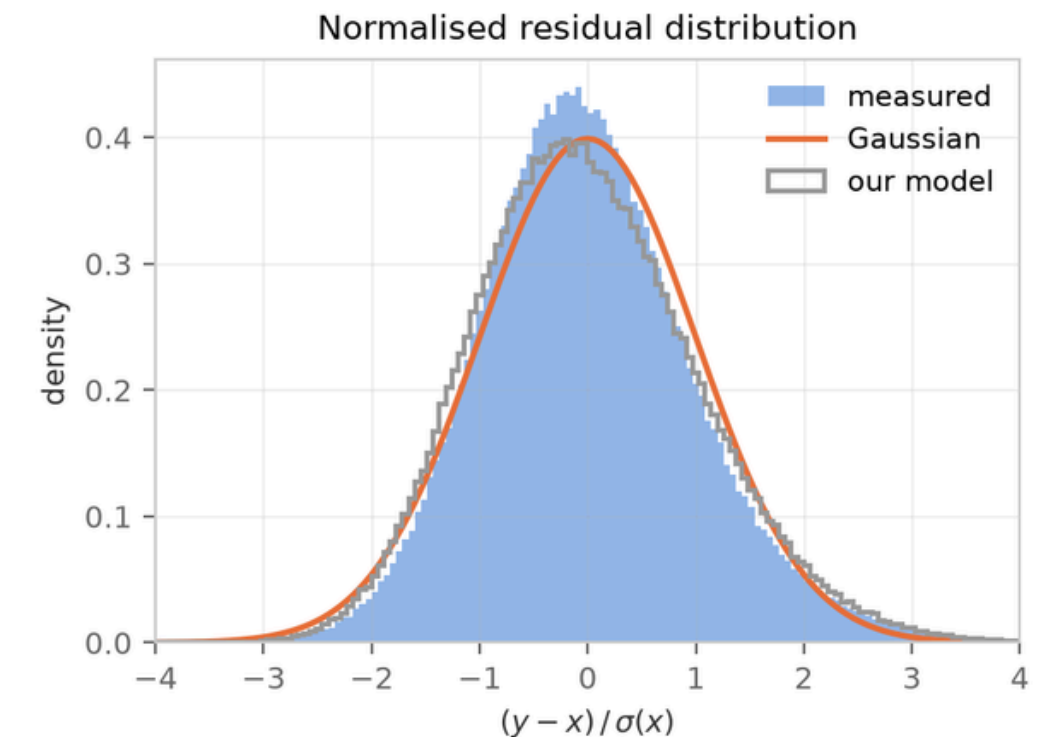
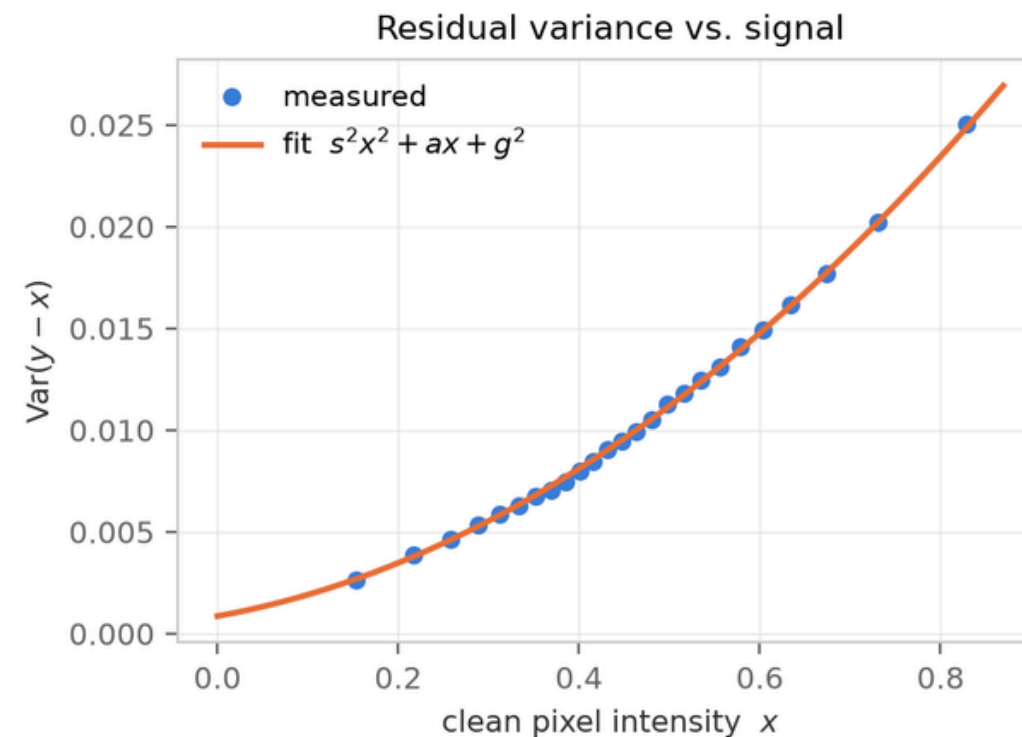
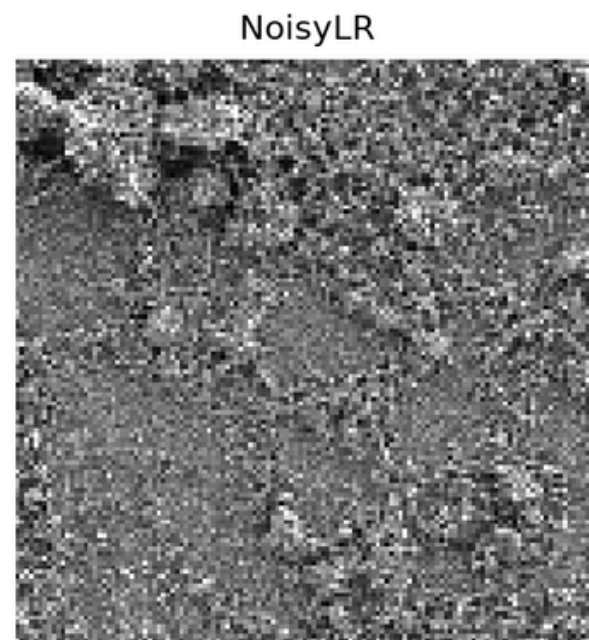
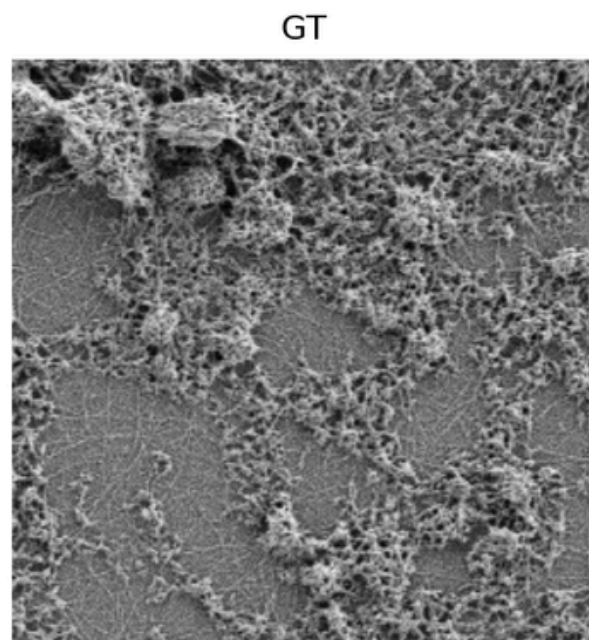
Dataset analysis and degradation observations

The Noise Formula

$$\text{Var}(y|x) = \underbrace{s^2 \cdot x^2}_{\text{speckle}} + \underbrace{a \cdot x}_{\text{shot}} + \underbrace{g^2}_{\text{read}}$$

$y = \text{NoisyLR}(128 \times 128)$
 $x = \text{Ground Truth}(256 \times 256)$

Fitted from real pairs, not assumed. $s = 0.1590$ (Gamma), $a = 0.008018$ (Poisson), $g = 0.0293$, $y = \text{NoisyLR}$, $x = \text{GT}$



Three terms, not two

Adding the Poisson shot term took the variance fit from R^2 **0.9963** $\rightarrow$ **0.9995**

At dark pixels shot noise DOMINATES and roughly half of every image is near-black. The old two-term fit absorbed it by inflating the other two.

Speckle is Gamma, not Gaussian

Variance identifies a model only to second moment. The shape was wrong. Real residuals are skewed and heavy-tailed at low intensity. Gamma carries the right positive skew, the standard coherent-speckle model.

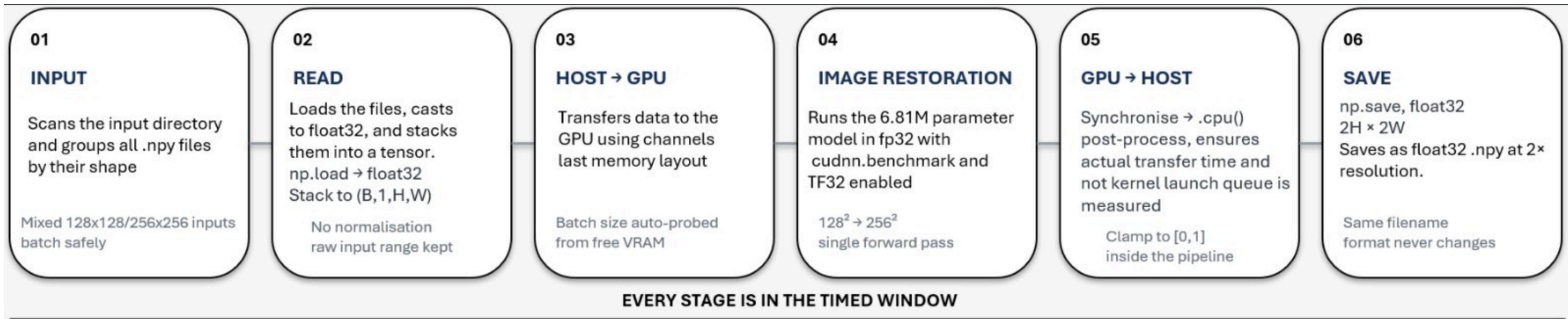
Level varies per image

$p90/p10 = 1.30\times$

(sd 0.113, full range 3.65 $\times$)

One set of weights must average over that spread, the argument for noise conditioning.

End-to-end pipeline



Throughput Optimization

Single forward pass - no TTA

8× dihedral re-measured: +0.086 dB PSNR but **LPIPS degrades 39%**, for 8.8× the compute.
Rejected.

No ensemble - one model

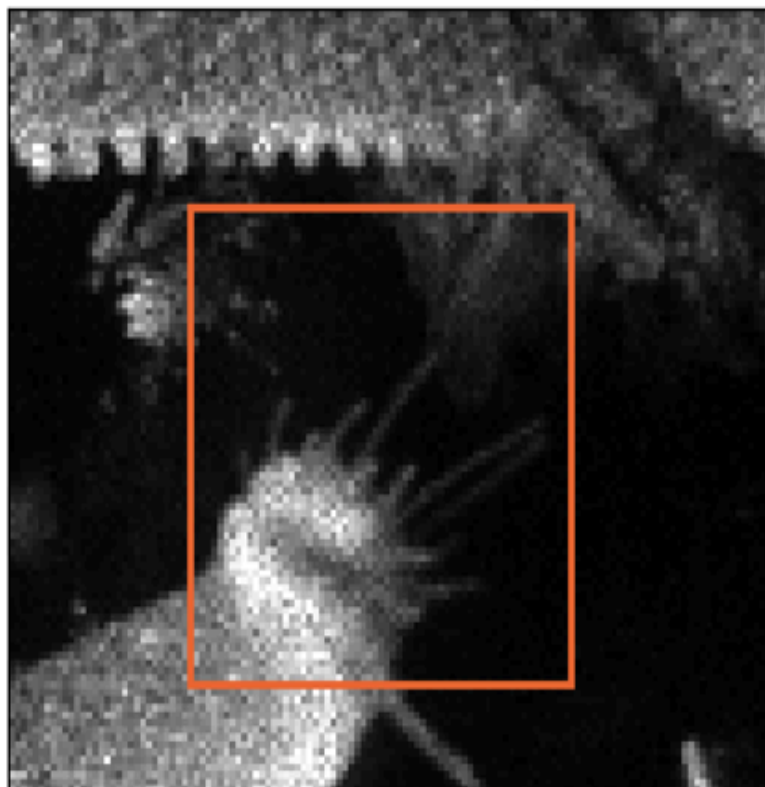
Averaging our three 36-epoch models moved every metric **inside the 0.006 dB**, for 2-3× the cost.
Rejected.

ADAPTIVE cudnn.benchmark

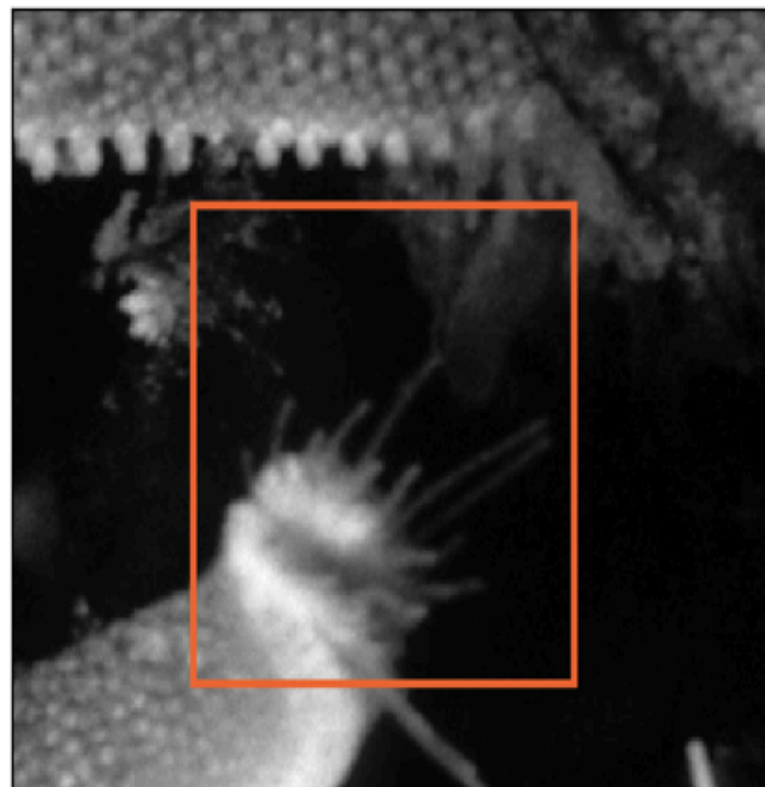
Autotuning finds 11%-faster kernels but costs 26.5 s up front, it repays only at **~9,650 images**.
Chosen (from the input count at runtime.)

Preprocessing, paired loader & upsampling experiments

NoisyLR 128×128
random 64×64 crop



GT 256×256
the exactly-corresponding 128×128



Paired data loader

Each sample loads a degraded 128×128 NoisyLR image with its exact 256×256 GT partner. Random crops are taken in strict 2× correspondence, preserving pixel alignment and the LR↔HR relationship throughout training.

Bicubic preprocessing

The 128×128 input is bicubic-upsampled to 256×256 first. MiniRestormer predicts a residual on top of this strong baseline, so the network focuses capacity on removing degradation and restoring detail.

DWT preprocessing - tried

DWT maps 128×128 → 4×64×64 (LL, LH, HL, HH), forcing Transformer processing at reduced resolution. With dim=32 and 16×16 bottlenecks, high-frequency details were smoothed, producing a soft 128×128 reconstruction.

Why DWT failed

DWT is theoretically lossless, but inaccurate sub-band prediction blurred the IDWT output. L1 loss and high-frequency noise suppressed LH/HL/HH details, which PixelShuffle could not recover.

PixelShuffle vs DWT capacity–resolution trade-off

PixelShuffle kept the network's spatial grid at 128×128 through the main encoder/decoder path and used one learned PixelShuffle at the end to reach 256×256. DWT traded spatial resolution for compression; with limited capacity, the model could not preserve high-frequency information well enough. The fixed IDWT then compounded the error before final upsampling.

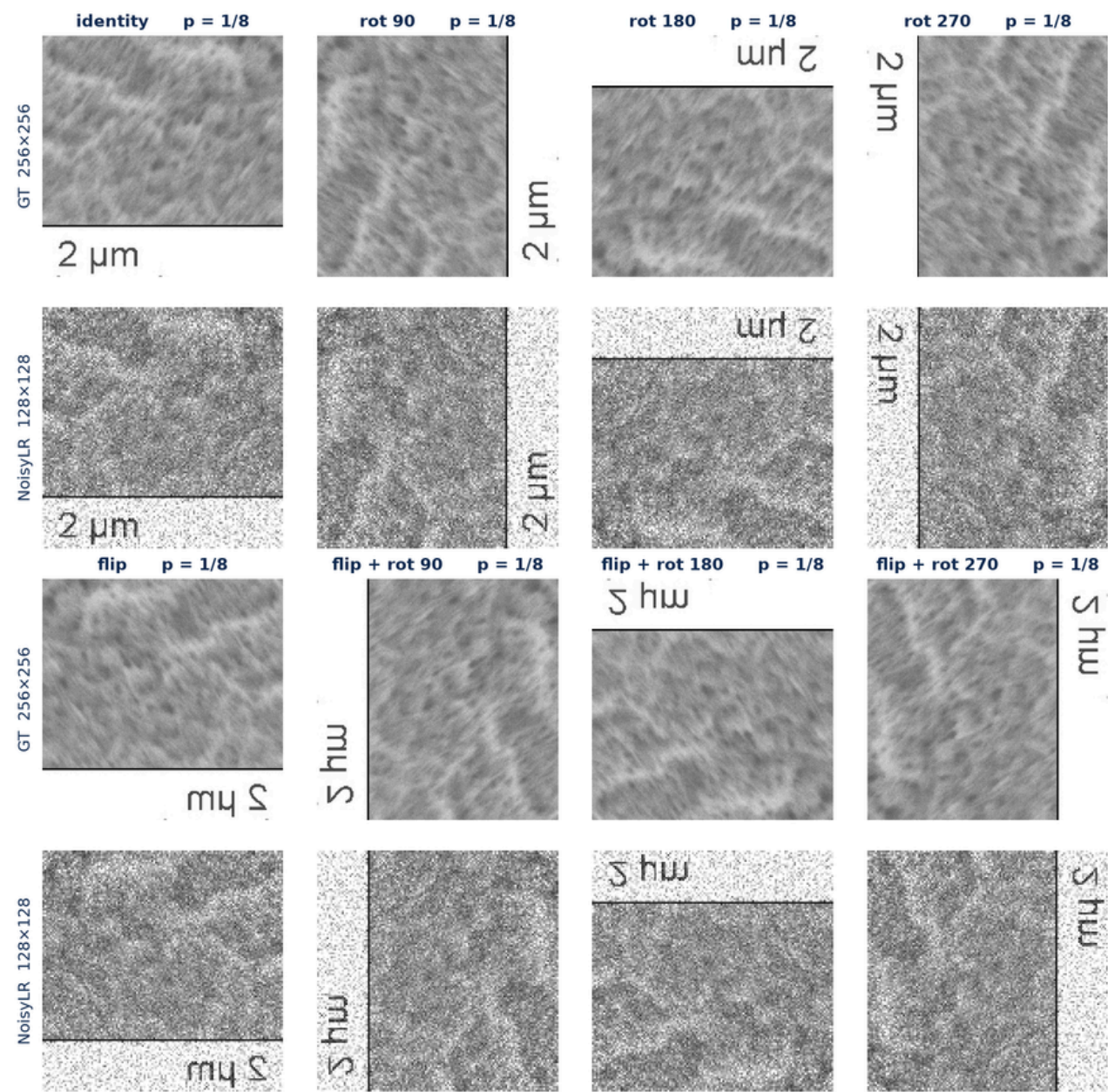
Augmentation

Geometric Augmentation

4 rotations × 2 flips × random crop. Exact pixel permutations, no interpolation, so 2× correspondence and speckle statistics stay intact.

Geometric augmentation: the 8 dihedral transforms

Applied identically to the GT and its NoisyLR partner, drawn uniformly per sample. Exact pixel permutations — no interpolation, so the 2× correspondence and the speckle statistics stay intact.



Synthetic pairs from GT : Not used

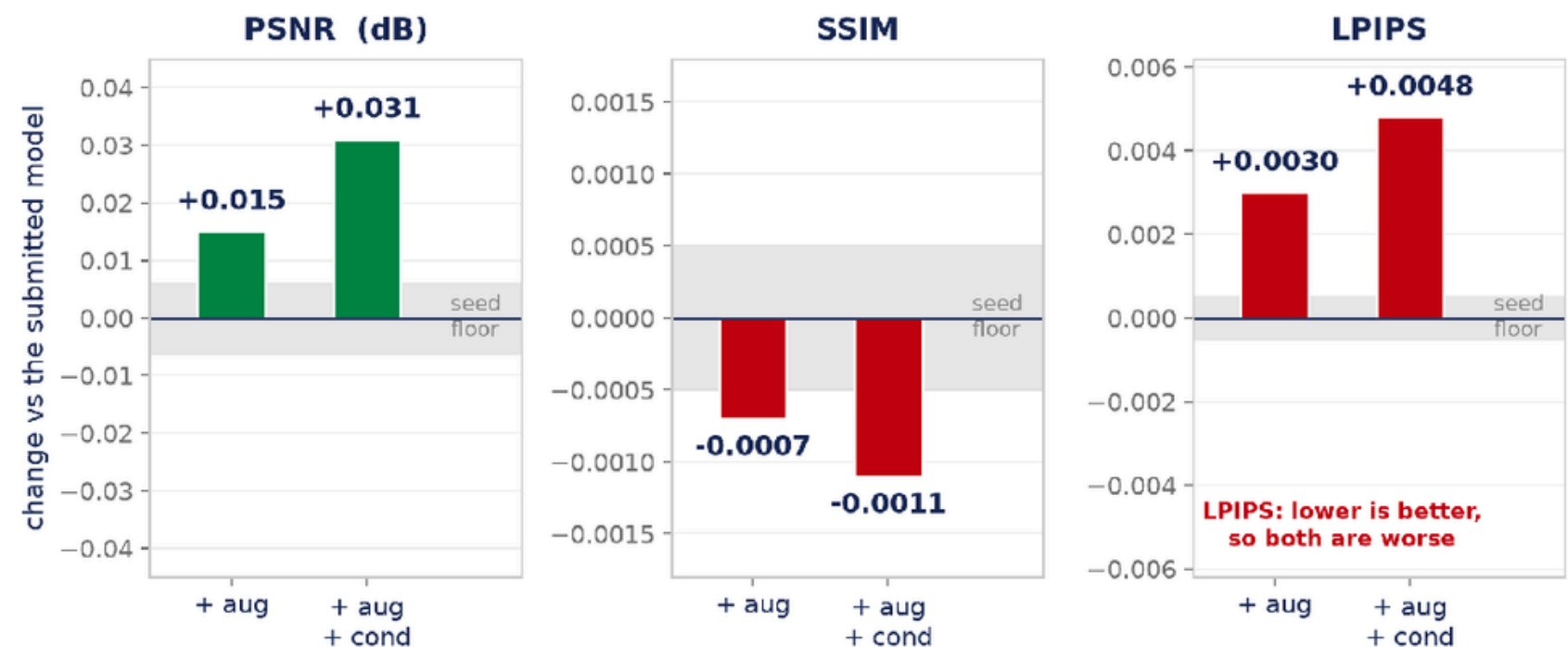
Replacing the input with one synthesised from GT, across three successively better forward models:

-0.048 dB / -0.0063 SSIM / +0.0092 LPIPS vs model using no synth pairs
The model got measurably better each round but the sign never flipped.
The residual is measured as $\text{real_NoisyLR} - \text{bicubic_down}(\text{GT})$. That assumes GT is clean and it isn't, as the down sampling also eats some of the info of the image.

Degradation augmentation: Not used

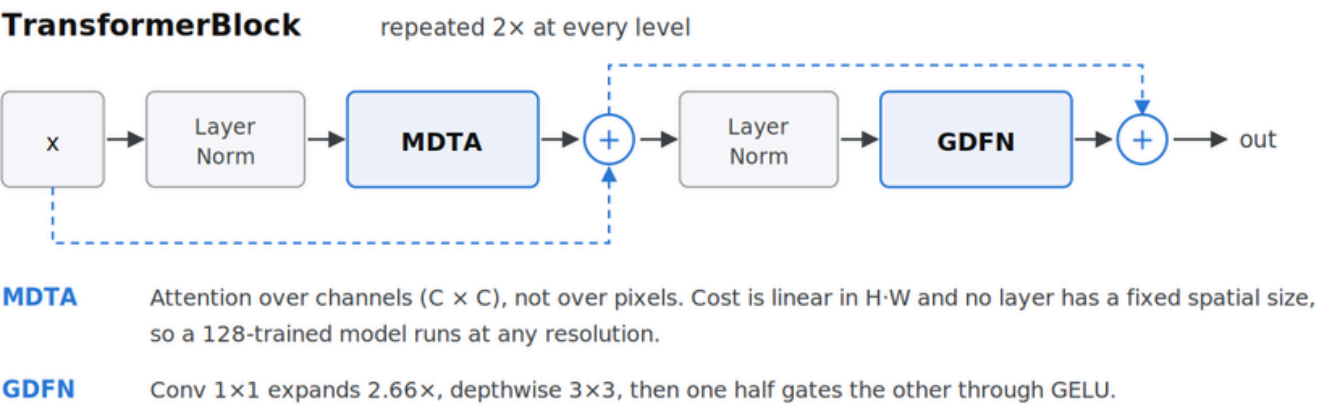
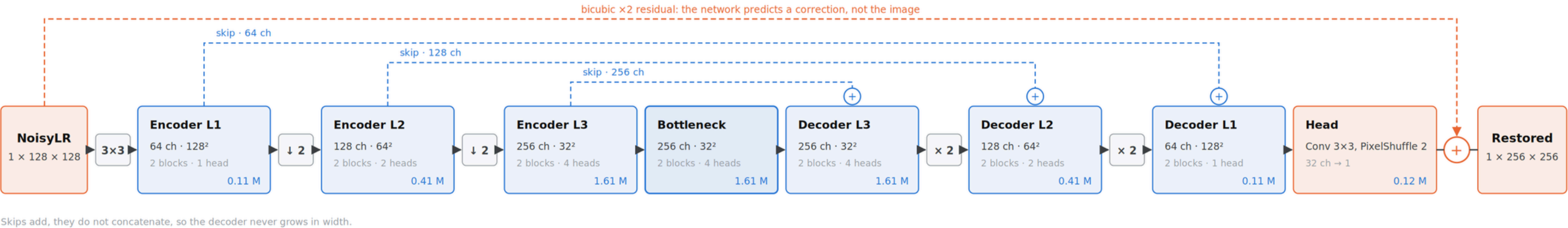
New in Phase 2: varies HOW NOISY the input is. Artificial noise is added onto existing NoisyLR.

+0.031 dB / -0.0011 SSIM / +0.0048 LPIPS, even though we get a increase in PSNR, the SSIM and LPIPS gets hit (In Std-dev terms, we lose 10 std of LPIPS to gain 5 std of PSNR, which we felt was not worth it)



Model architecture and design rationale

MiniRestormer 6.81 M parameters, $1 \times 128 \times 128 \rightarrow 1 \times 256 \times 256$
dim 64 · blocks (2,2,2) · heads (1,2,4) · wide head 32 · additive skips · 2× PixelShuffle · global bicubic residual · fp32



Parameter budget

stage	resolution	params	share
Encoder L1, L2	$128^2, 64^2$	0.52 M	7.6%
Encoder L3	32^2	1.61 M	23.6%
Bottleneck	32^2	1.61 M	23.6%
Decoder L3	32^2	1.61 M	23.6%
Decoder L2, L1	$64^2, 128^2$	0.52 M	7.6%
Down and up resampling		0.82 M	12.0%
Input conv and head	$128^2 \rightarrow 256^2$	0.12 M	1.8%
Total		6.81 M	100%

71% of the parameters sit at 32×32 , where they are cheapest to run.

Design Choices

Bicubic residual skip

The network predicts a correction, not 65k pixels from scratch. Training starts near the baseline instead of at noise.

Attention over channels, not pixels

MDTA's attention matrix is $C \times C$, so cost is linear in $H \cdot W$ and no layer has a fixed spatial size a 128-trained model runs at 256 or 512 unchanged.

Additive skips, not concatenation

Keeps the decoder at encoder width. This is where the parameter saving over full Restormer comes.

Wide reconstruction head

The original head squeezed 64 channels through a 1-channel conv at full resolution. Widening it to 32 removed that bottleneck.

Loss functions and training setup

$$L = 0.50 \cdot \text{Charbonnier} + 0.35 \cdot (1 - \text{SSIM}) + 0.15 \cdot \text{gradient} + 0.10 \cdot \text{LPIPS}$$

Loss	Weight	Description
Charbonnier Loss	0.50	Smooth L1. Robust to the speckle outliers that make plain L2 chase individual bright pixels.
1 – SSIM	0.35	Structural term, but it also rewards blur, so it is down-weighted as PSNR also rewards it.
Gradient	0.15	Penalises edge softening directly, pushing against the two blurring terms above.
LPIPS	0.10	A scored metric that rewards texture. Cut perceptual distance 43% for 0.02 dB of PSNR, the best trade-off we found.

Checkpoint selection uses **SSIM – LPIPS**, never PSNR

TRAINING SETUP

optimiser	AdamW, wd 1e-4
schedule	3-epoch warmup → cosine to 1e-6
LR / batch	1e-4 / 6 (auto-probed)
precision	fp32 — fp16 diverged
stabilisers	EMA 0.999, grad clip 1.0, spike reject
split	90/10, seed 42, held out from training and selection

SSIM

Our loss: **11×11 Gaussian, zero-padded border**.
Scored: **7×7 uniform, sample covariance, border cropped**.
The two functions differ by **0.024 SSIM** though they **correlate at 0.9966**, when trained on the exact scored metric: **+0.0005 SSIM for -0.050 dB PSNR**.

HF Ratio

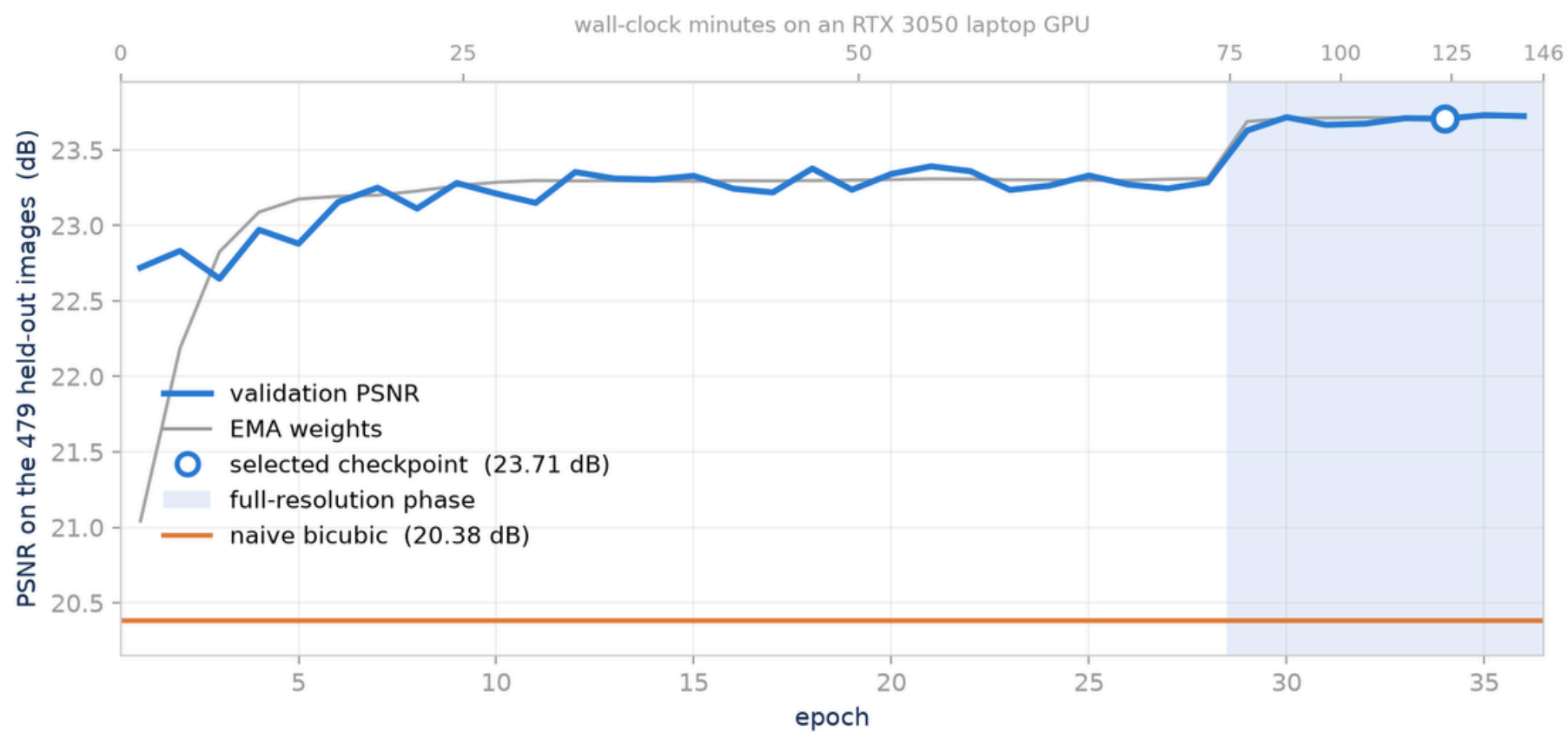
A fourth, UNSCORED diagnostic: hf_ratio, high-frequency energy above 0.25 cycles/px relative to GT. Its only job is to catch the blur that both headline metrics reward.

Frequency-domain loss (mentor advice)

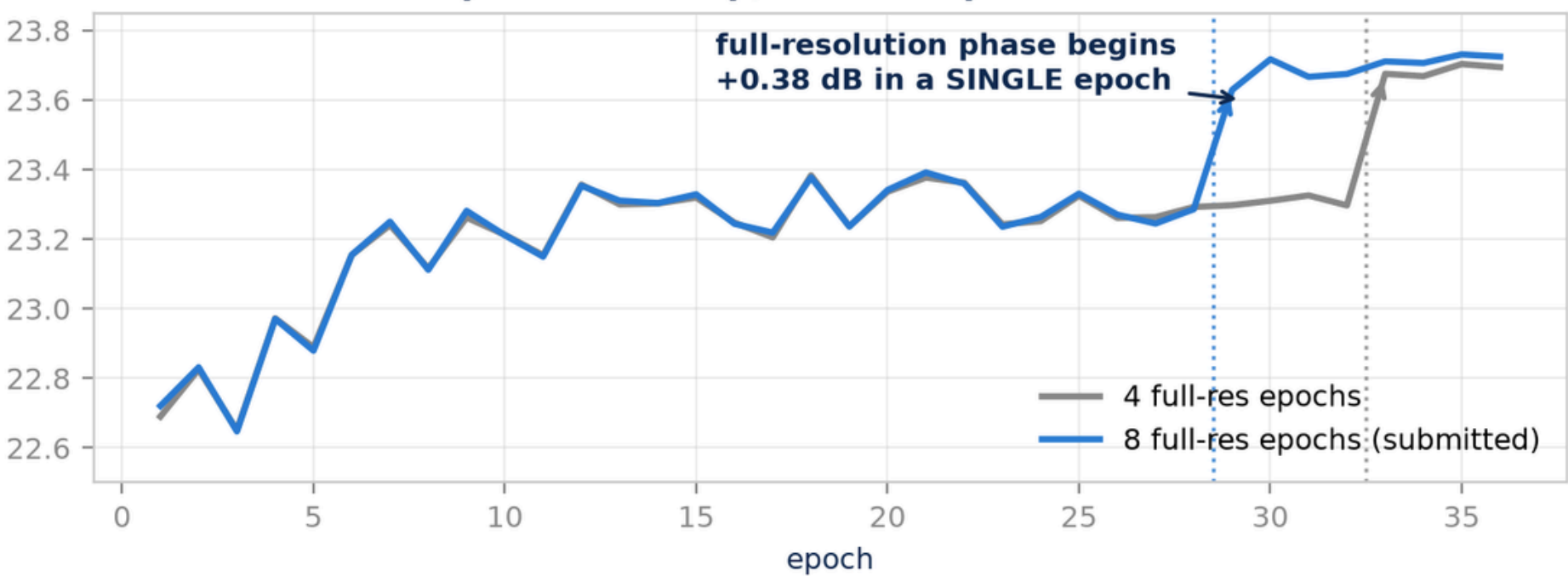
L1 on log1p|FFT| lifted high-frequency retention **0.470 to 0.589** - the largest move on that quantity in the project, but **LPIPS did not change** for **0.111 dB reduction of PSNR**.

Experiment tracking and baseline comparison

MiniRestormer validation PSNR



The full-resolution phase is a step, not a ramp



Variable	PSNR	SSIM	Verdict
full-res phase fires	0.377	0.0127	BIGGEST WIN
FULLRES_EPOCHS 4 -> 8	0.038	0.0023	adopted
EPOCHS 11 -> 36	0.059	0.0045	adopted
synthetic data 25%	-0.048	-0.0063	rejected
NOISE_COND alone	-0.007	-0.003	rejected
TTA 8x	0.086	0.0038	LPIPS +39%
ensemble model	0.008	-0.0003	1/3 of throughput

PSNR, SSIM and LPIPS results

Every tries against the shipped model on the 479 held-out images

RUN	PSNR	SSIM	LPIPS	Change vs SUBMITTED
Submitted	23.706	0.6230	0.1577	–
Aug+Cond	23.737	0.6219	0.1625	+0.031*/ –0.0011*/ +0.0048*
Augment	23.721	0.6223	0.1608	+0.015*/ –0.0007*/ +0.0030*
4 full-res	23.668	0.6207	0.1599	-0.038*/ –0.0023*/ +0.0022*
Noise_Cond	23.662	0.6200	0.1602	+0.0045*/ –0.0030*/ +0.0025*
11 epochs	23.607	0.6160	0.1737	-0.099*/ –0.0070*/ +0.0043*
Phase 1 arch	23.306	0.6120	0.1643	+0.362*/ –0.0087*/ +0.0043*

On the Test Dataset

PSNR: 23.982 dB (+0.276)
SSIM: 0.6455 (+0.225)
LPIPS: 0.1509 (-0.0068)

Why we ship this one

It is the only model that is **best on all three metrics at once**.
With the metric weights undisclosed, that is the only robust position to hold - every challenger wins one metric by trading another.

The augmented + conditioned variant wins **PSNR by 0.031 dB** but costs **0.0048 LPIPS**

Runtime, batch size and optimisation

val data inference – 25.50 ms/img
end-to-end for 479 images, measured
start to finish on a 4 GB dev GPU. No
TTA, no ensemble.

test data inference – 24.82 ms/img
end-to-end for 297 images, measured
start to finish on a 4 GB dev GPU. No
TTA, no ensemble.

N	off	on	on/off
128	3.66	29.80	8.14x
479	12.70	42.69	3.36x
1916	48.36	63.10	1.30x
5748	140.66	153.16	1.09x

off = 1.04s + 24.33 ms/img on = 27.6s + 21.58 ms/img
crossover ~9,650 images
BENCHMARK_MIN_IMAGES = 1000

BATCH SIZE DIFFERENT FOR TRAIN AND INFER

- **ms/img** is flat from **batch 1 to 16** and the CPU to GPU **launch gap** is **nearly 0** for a single 128×128 image already saturates a **20 W RTX3050 Laptop GPU**.
- CUDA graphs and kernel fusion would gain nothing here. On an **H100** this inverts, it will be launch-bound at small batch, and **auto_batch's cap of 256** is the one speed parameter we are genuinely hoping to carry us.

Test set – 4GB GPU

Stage	Duration (s)
import+init	1.33
build model + read	0.35
inference	7.37
write flush	0
TOTAL	9.06

No torch.compile

30 - 60 s of compile time to save well under 1 s of compute. Ruled out by arithmetic, not tried and abandoned.

Nothing heavy is imported

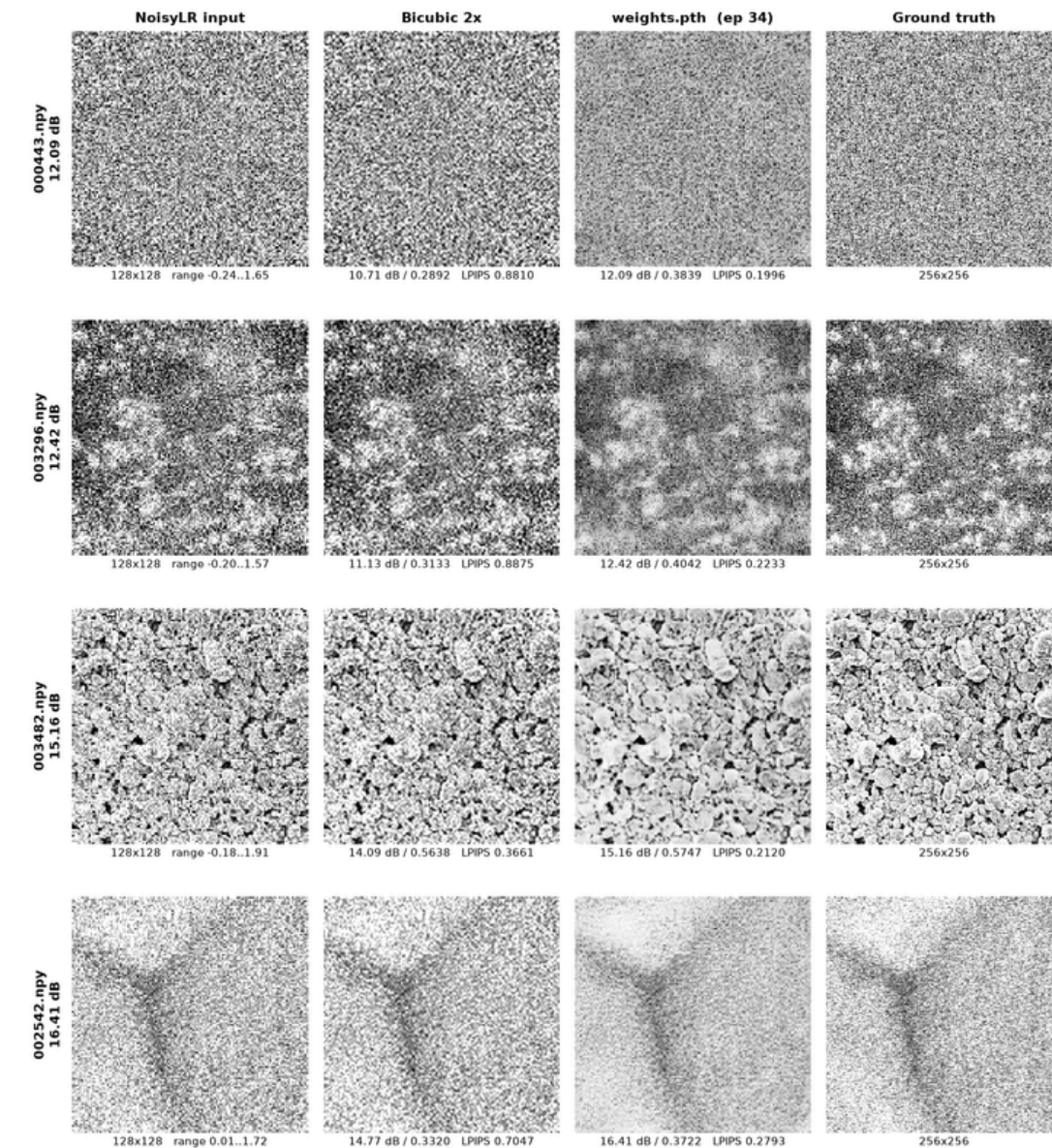
No torchvision, lpips or skimage in the inference path, they cost **1 - 2 s** of startup and are training and scoring dependencies only.

Resolution-agnostic by construction

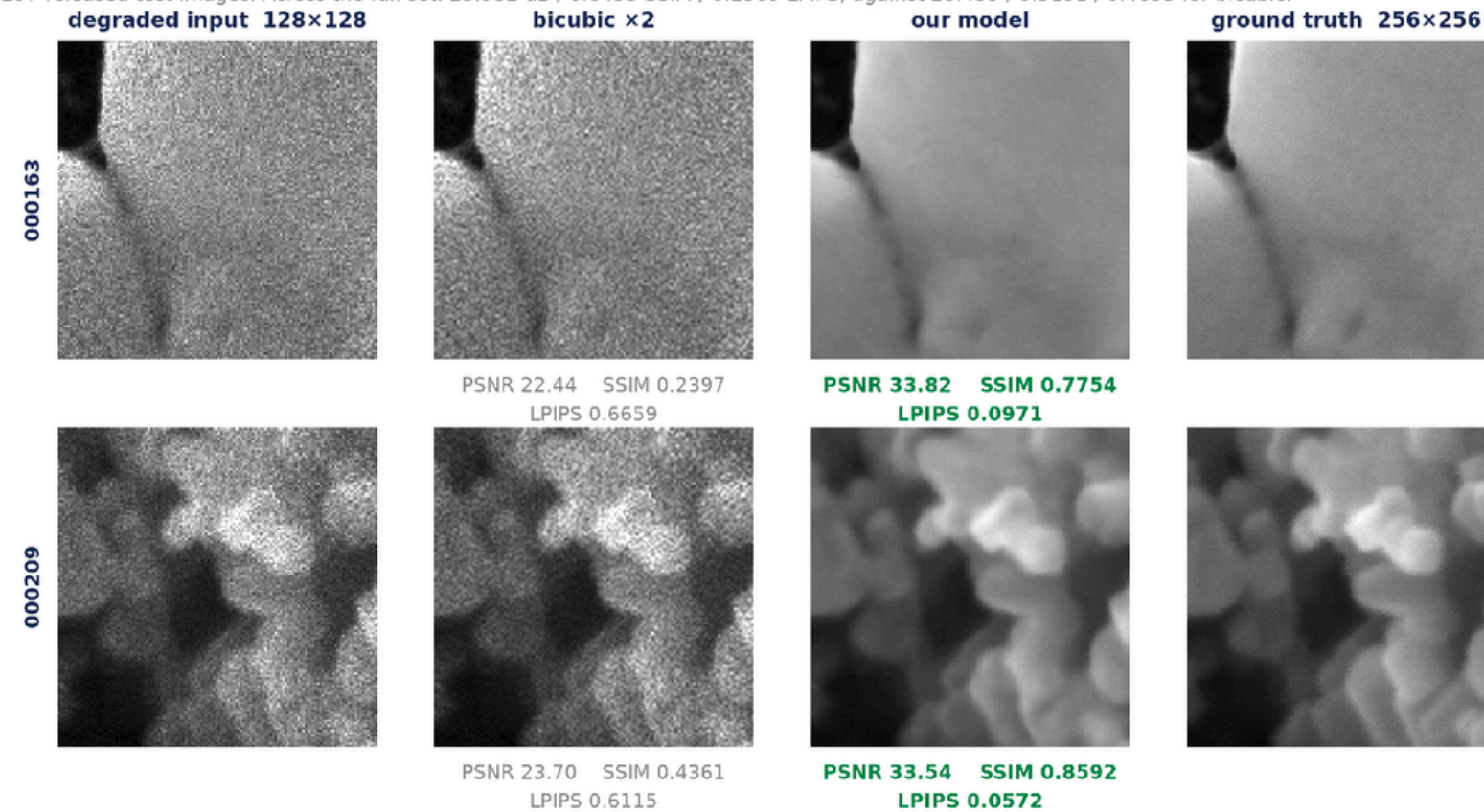
Inputs are reflect-padded to a multiple of 8 and cropped back; images are grouped by shape so 128 and 256 inputs never share a padded batch.

Visual results, failure cases and limitations

Held-out validation — worst 4 of 479

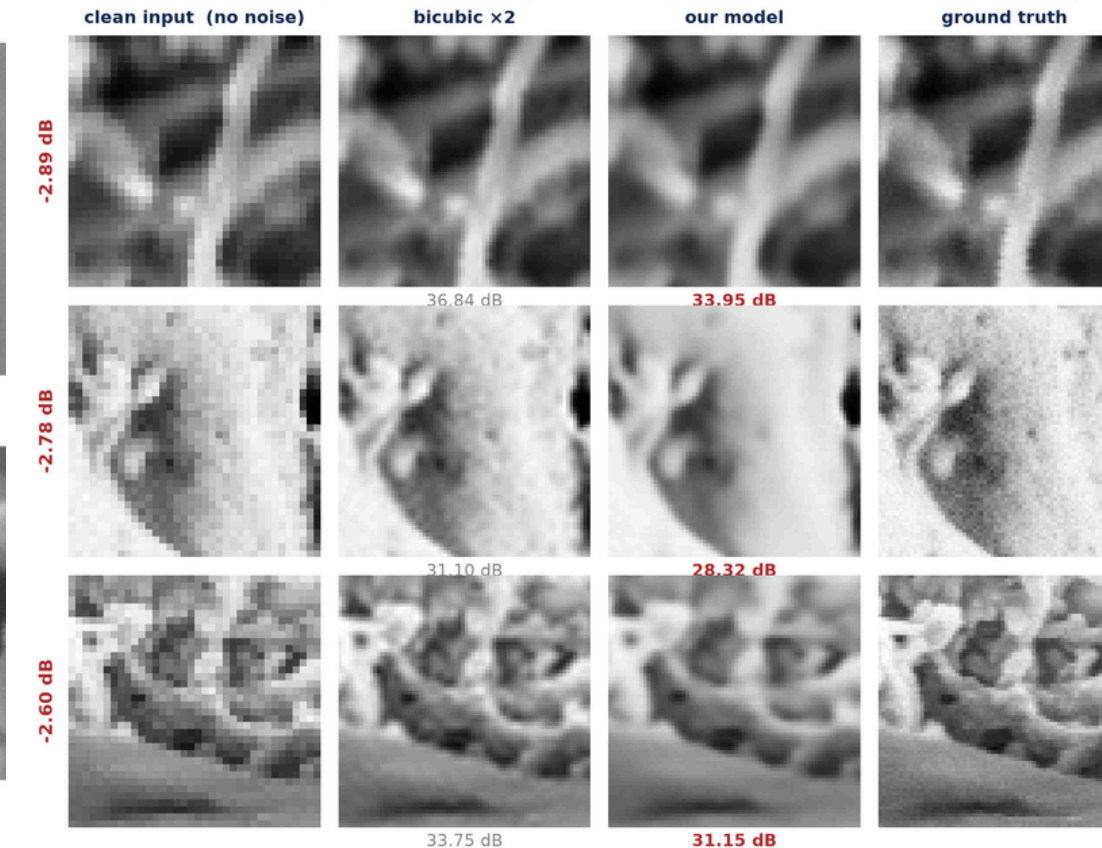


297 released test images. Across the full set: 23.982 dB / 0.6455 SSIM / 0.1509 LPIPS, against 20.455 / 0.5191 / 0.4655 for bicubic.



Failure case: given a clean input, the model scores below plain bicubic

The network has only ever seen noisy inputs, so it denoises regardless — erasing detail that was never noise. It loses on 99% of images.



The worst cases

- Our four lowest-PSNR images look like salt-and-pepper. Measuring lag-1 autocorrelation of the GT, only **one of the four is genuinely near-noise and dataset-wide only 3 of 4,785**.
- The rest is real, very fine-grained SEM. **We still beat bicubic on those images by 1.38 dB and LPIPS 0.1996 vs 0.8810.**

Honest limitations

- We retain only **~47% of GT's high-frequency** energy.
- Given a **CLEAN input** the model scores 1.0 dB(on avg) **WORSE** than plain bicubic
- It applies one fixed denoising strength and cannot back off. Four runs across two queues moved that by 0.09 dB. It is our largest known weakness.

Where it fails

- When speckle destroys a texture completely, the loss-optimal answer is the average of every texture that could have been there and that average is a blur. Fundamental limitation of supervised learning on this data.
- **Dropping the worst 24 val images moves the mean 23.706 to 24.054 dB**

Conclusion, external resources and repository

What we are submitting

- A single 6.81 M-parameter MiniRestormer, one forward pass per image, 23.706 dB / 0.6230 SSIM / 0.1577 LPIPS on 479 images held out from training and from model selection
- 55% of the measured PSNR headroom and 71% of the SSIM headroom, against a floor and a ceiling we measured rather than assumed
- The only configuration that is best on all three metrics at once is chosen over a higher-PSNR variant and over a more noise-robust variant, in both cases because the metric weights are undisclosed
- 13.1 s end-to-end for 479 images, of which ~1.8 s is irreducible process startup

External resource disclosure

Resource	How it is used
https://arxiv.org/abs/2204.04676	nafnet
https://arxiv.org/abs/2111.09881	restormer
https://ieeexplore.ieee.org/document/1005694	deep learning
lpips (Zhang et al. 2018), AlexNet backbone	training loss + reported metric. NOT in the inference pipeline
PyTorch, NumPy, scikit-image, SciPy	framework, metrics

Repository

https://github.com/KAlcorpse/Git_Push_and_Prayer_Phase_2

Reproducibility

Seed 42 · 90/10 split held out from training and selection · every hyper-parameter overridable by environment variable, so no experiment required a source edit · per-epoch train_log.csv for every run · the checkpoint's full architecture is recovered from its weight shapes at load time, so inference cannot silently build the wrong model.

CLOSING NOTE

THANK YOU AKSHAT SINGH SIR, FOR GIVING VALUABLE ADVICES FOR THIS PHASE OF THE HACKATHON.